

[Menu](#)

Core Components

▼ Relevant source files

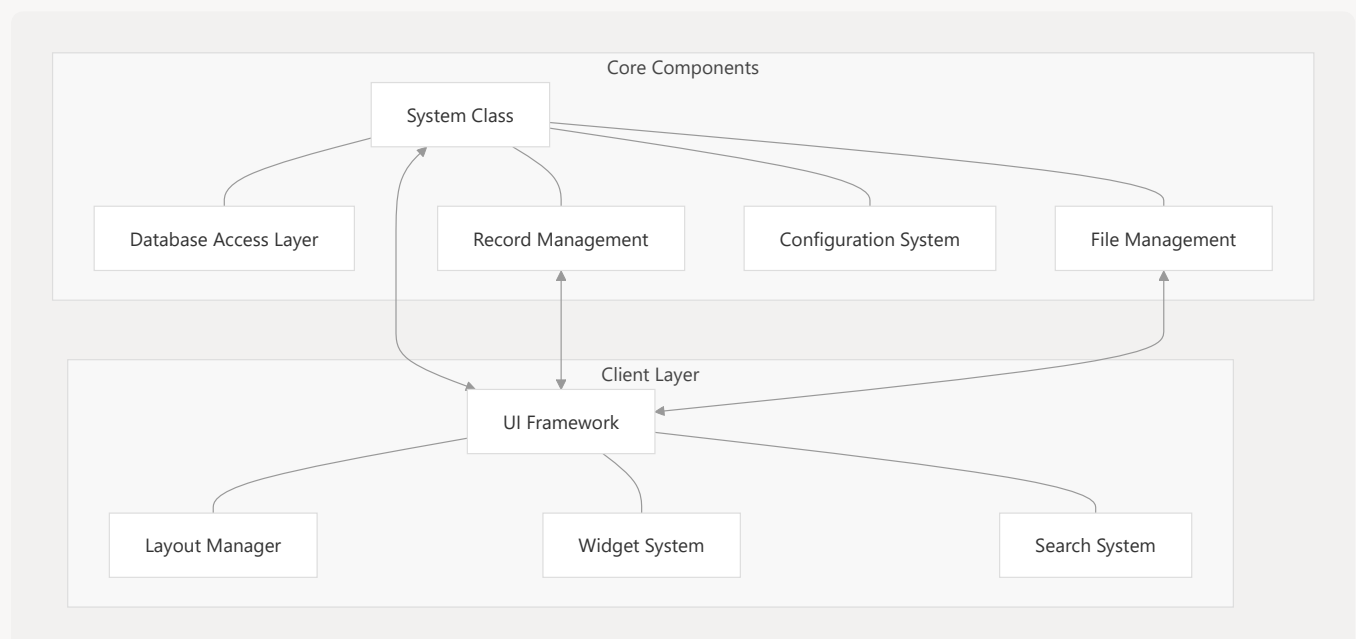
`admin/utilities/downloadInteractionLog.php``hclient/core/layout.js``hclient/framecontent/initPage.php``hclient/widgets/entity/manageRecUploadedFiles.js``hclient/widgets/entity/searchRecUploadedFiles.html``hclient/widgets/entity/searchRecUploadedFiles.js``hserv/System.php``hserv/consts.php``hserv/controller/usr_info.php``hserv/dbaccess/utils_db.php``hserv/records/edit/recordModify.php``hserv/records/edit/recordTitleMask.php``hserv/records/edit/recordsBatch.php``hserv/records/import/importAction.php``hserv/records/import/importParser.php``hserv/records/search/recordSearch.php``hserv/structure/dbsUsersGroups.php``hserv/structure/search/dbsData.php``hserv/utilities/utils_db_load_script.php``index.php``layout_default.js``redirects/resolver.php`

`viewers/record/viewRecord.php`

This document outlines the core components of the Heurist academic knowledge management system, explaining their roles and interactions. Understanding these fundamental building blocks is essential for developers working with the codebase. For configuration details, see [Configuration System](#).

System Overview

Heurist is built with a modular architecture centered around several key components that work together to provide the system's functionality. The core components include the System class, database access layer, record management, file management, and UI framework.



Sources: `index.php` 20-33 `hserv/System.php` 19-65

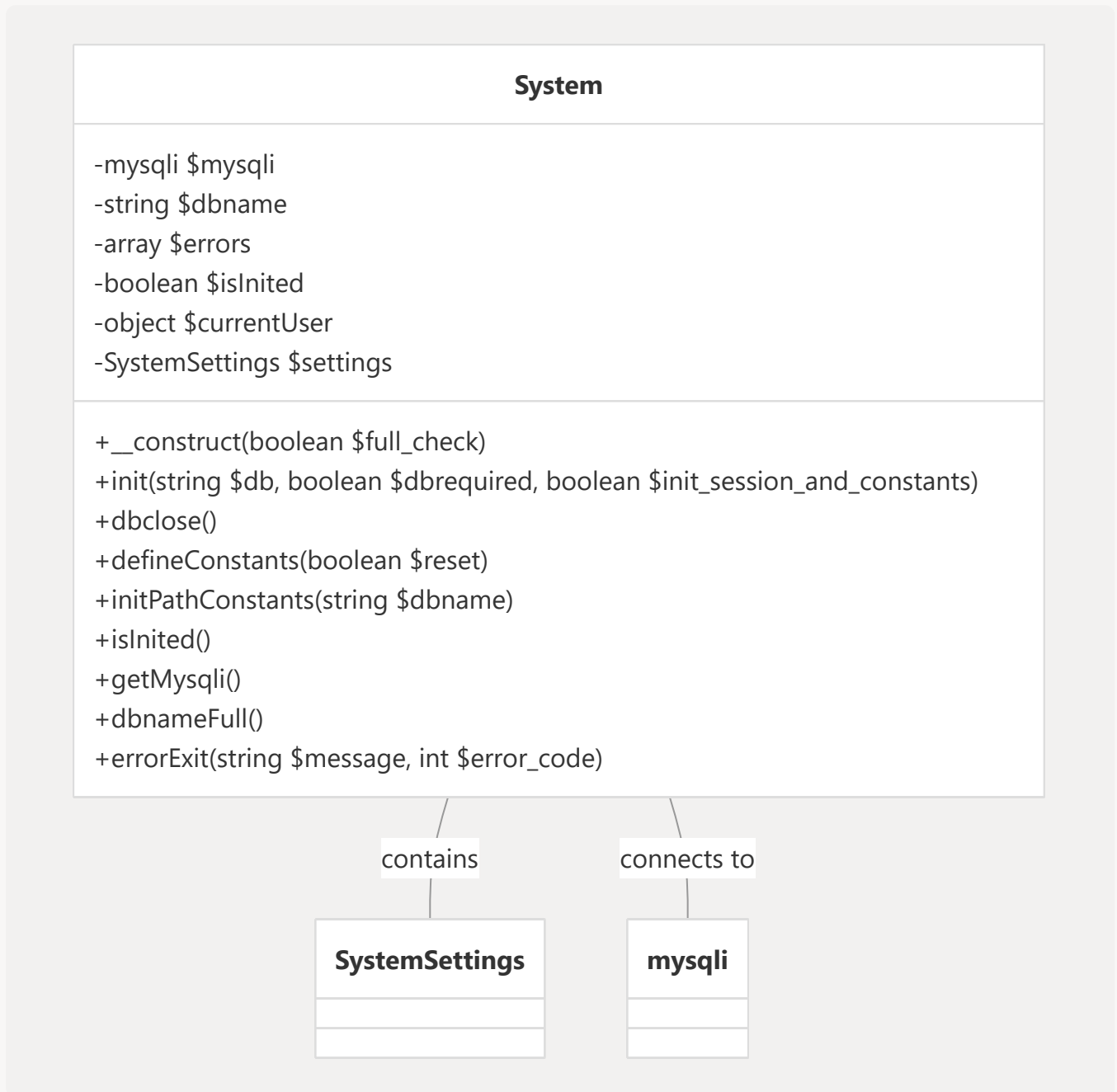
System Class

The System class (`hserv\System`) serves as the central coordinator for the Heurist application. It handles initialization, database connections, user authentication, file path management, and error handling.

Key Responsibilities

- Establishing database connections
- Authenticating users and managing sessions
- Defining system constants for record types and field types
- Managing file paths and URLs

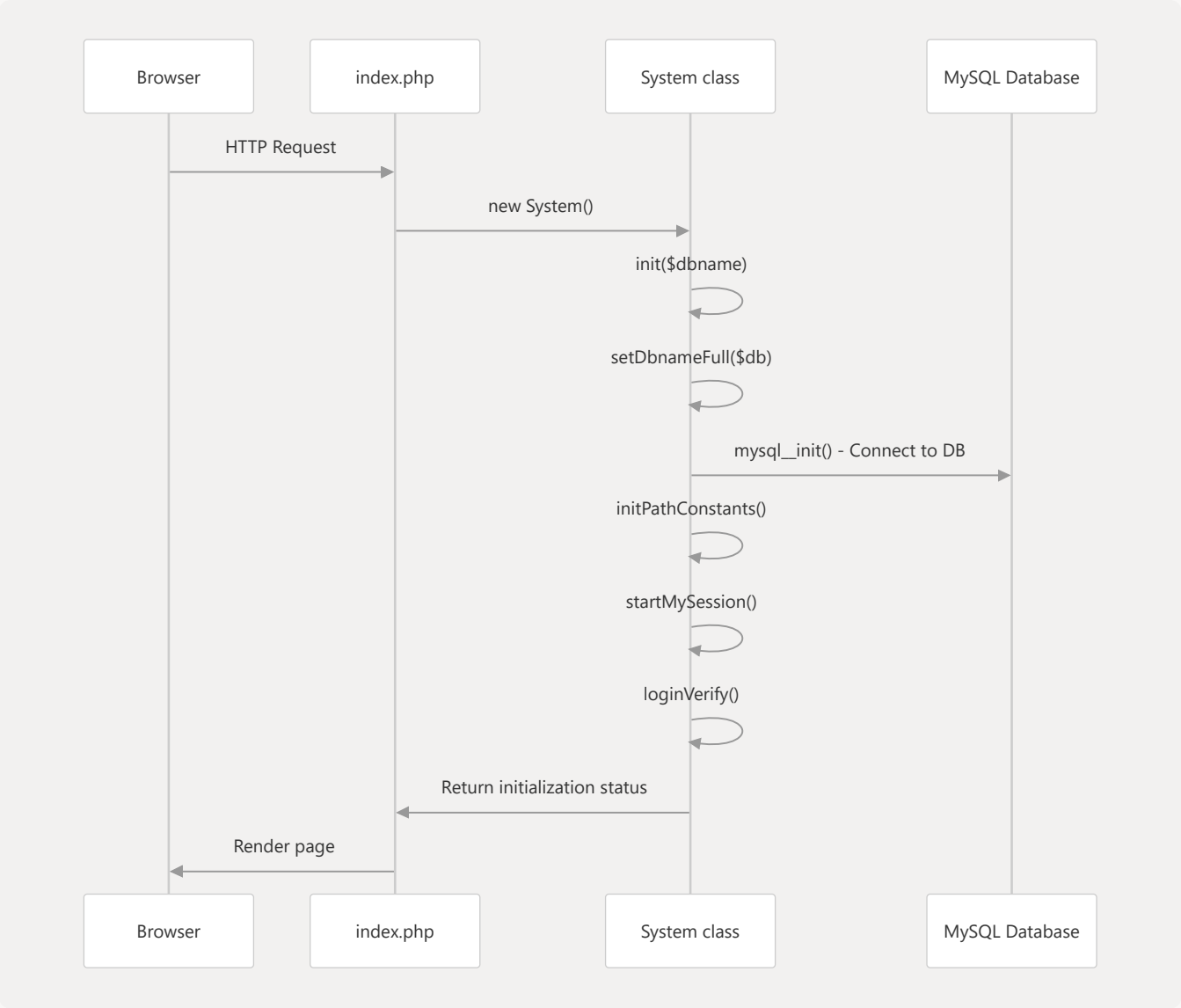
- Providing error handling and reporting
- Initializing system components



Sources: `hserv/System.php` 45-100 `hserv/System.php` 160-196

Initialization Process

The System initialization process is a critical sequence that sets up the environment for the application:



Sources: `index.php` 28-46 `hserv/System.php` 96-145 `hclient/framecontent/initPage.php` 45-70

Database Access Layer

The database access layer provides functions for interacting with the MySQL database. It's primarily implemented in `hserv/dbaccess/utills_db.php` and used throughout the system.

Key Database Functions

Function	Purpose
<code>mysql__init()</code>	Initializes database connection
<code>mysql__connection()</code>	Establishes connection to MySQL server
<code>mysql__usedatabase()</code>	Selects the database to use
<code>mysql__select()</code>	Base function for executing queries
<code>mysql__select_value()</code>	Returns first column of first row

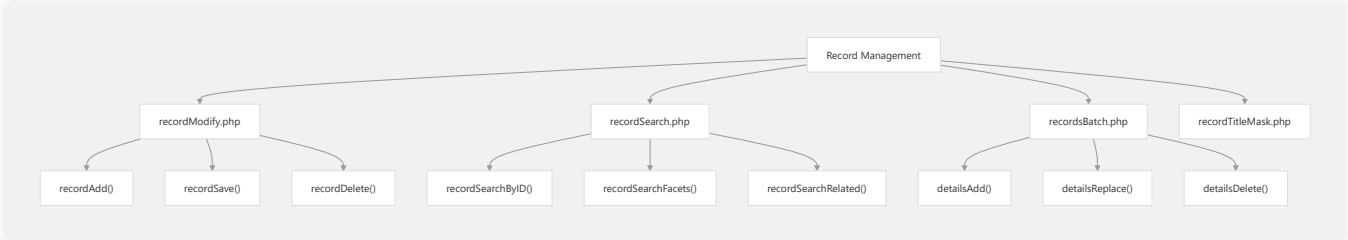
Function	Purpose
<code>mysql__select_assoc()</code>	Returns associative array of results
<code>mysql__begin_transaction()</code>	Starts a database transaction

Sources: `hserv/dbaccess/utls_db.php` 84-101 `hserv/dbaccess/utls_db.php` 111-138
`hserv/dbaccess/utls_db.php` 145-175

Record Management System

The record management system is the core functionality of Heurist, handling record operations like creation, updating, deletion, and searching.

Key Components



Sources: `hserv/records/edit/recordModify.php` 18-43
`hserv/records/search/recordSearch.php` 3-27 `hserv/records/edit/recordsBatch.php` 11-44

Record Operations

The record management system provides several key functions:

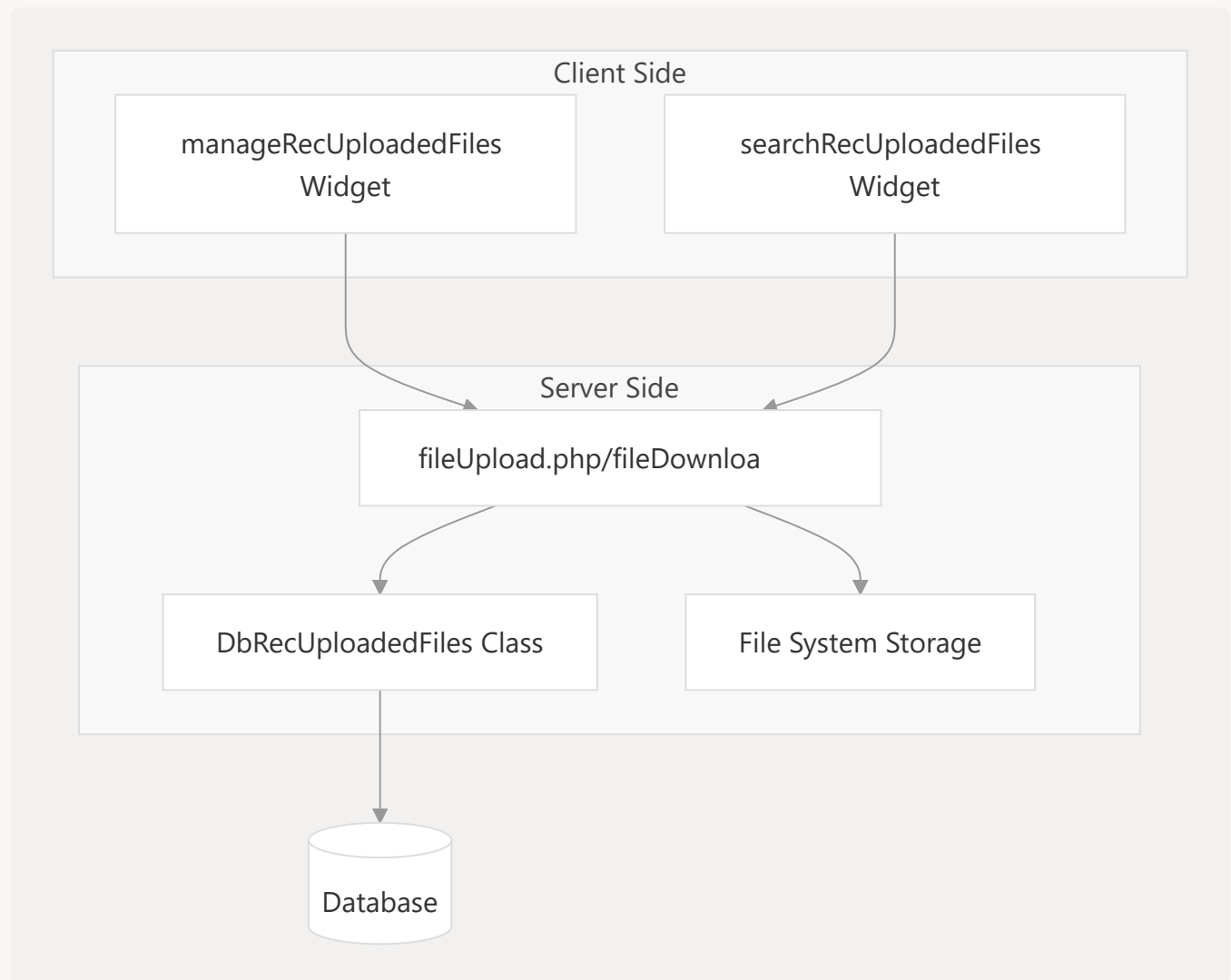
- `recordAdd()` : Creates new records with proper ownership and access rights
- `recordSave()` : Updates existing records with validation and detail processing
- `recordDelete()` : Removes records with permission checking
- `recordSearch()` : Finds records based on various search criteria
- `recordUpdateTitle()` : Updates record titles based on title masks
- Batch operations for modifying multiple records simultaneously

Sources: `hserv/records/edit/recordModify.php` 170-290
`hserv/records/edit/recordModify.php` 330-444

File Management

The file management system handles file uploads, storage, retrieval, and metadata management.

Architecture



Sources: `hclient/widgets/entity/manageRecUploadedFiles.js` 21-50

`hclient/widgets/entity/searchRecUploadedFiles.js` 20-47

File Management Widget

The `manageRecUploadedFiles` widget provides the user interface for:

- Uploading local files
- Registering external file references
- Managing file metadata
- Viewing file contents
- Linking files to records

The widget handles different file types and provides appropriate viewers for images, videos, documents, and other formats.

Sources: `hclient/widgets/entity/manageRecUploadedFiles.js` 98-305

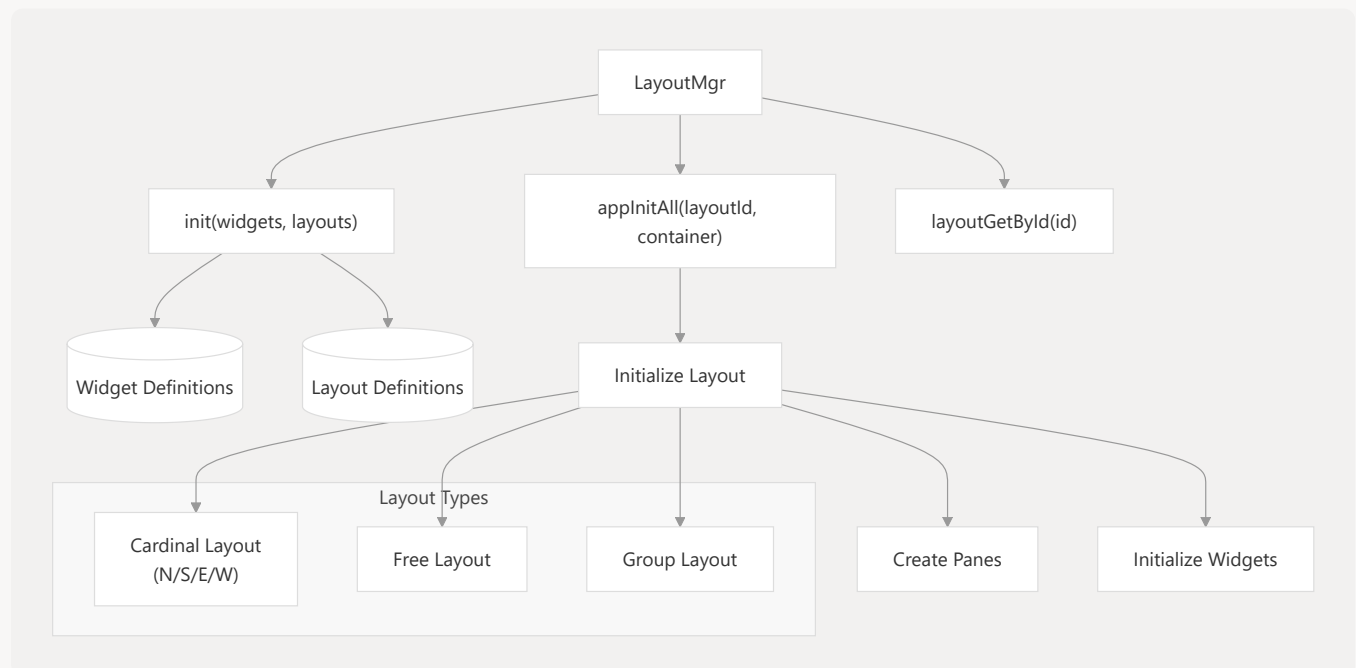
UI Framework and Layout System

The UI framework provides the user interface components and layout management for the Heurist application.

Layout Manager

The layout manager (`HLayoutMgr`) handles:

- Layout initialization
- Widget loading and initialization
- Layout configuration
- Responsive layout management



Sources: `hclient/core/layout.js` 29-126 `hclient/core/layout.js` 218-250

Widget System

The widget system provides UI components such as:

- Search widgets
- Record viewing and editing widgets
- Result list widgets
- Map widgets

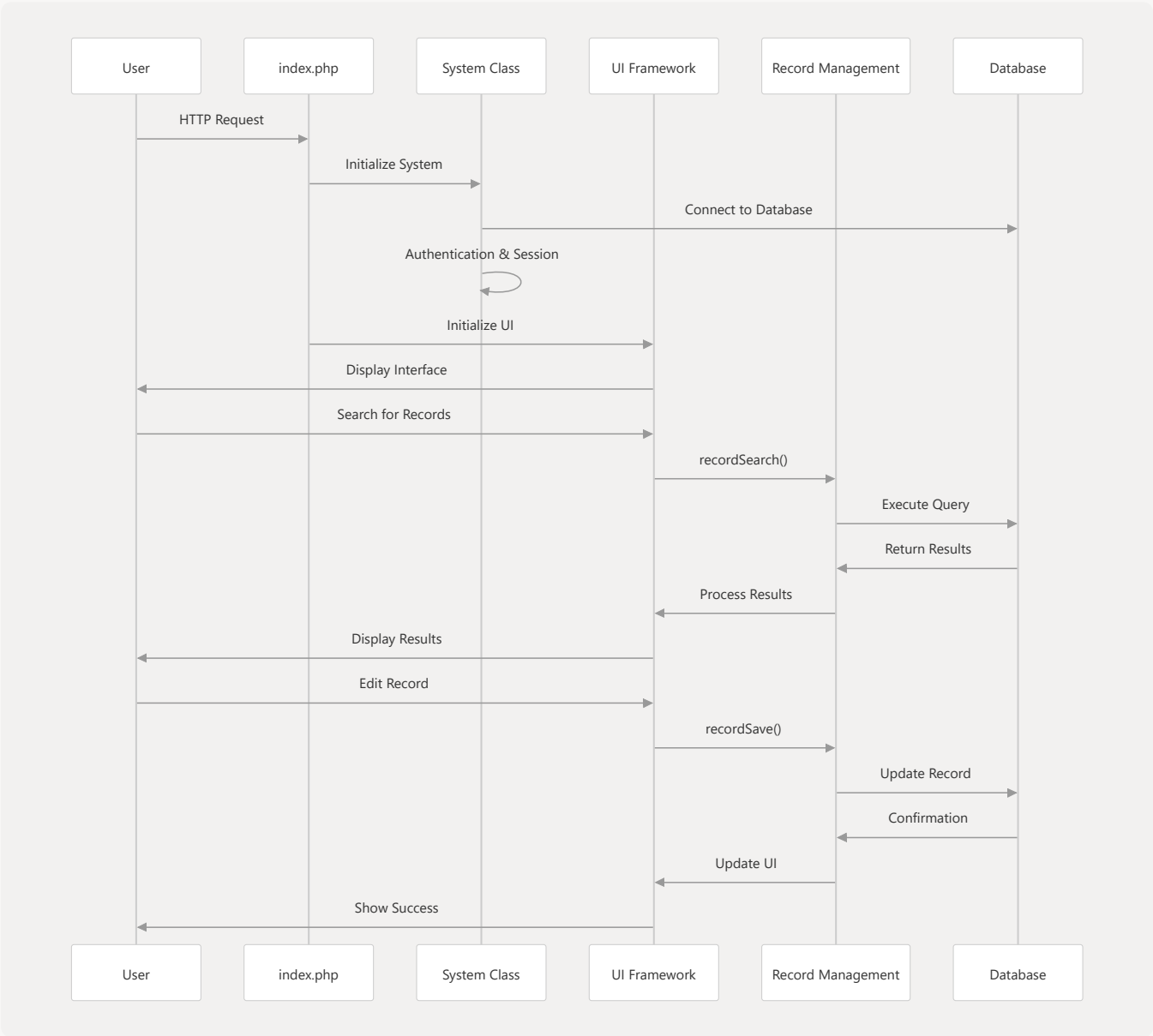
- File management widgets

These widgets are defined in `layout_default.js` and initialized by the layout manager.

Sources: `layout_default.js` 40-78 `layout_default.js` 98-246

Component Interactions

The core components interact in various workflows. Here's a typical user interaction flow:



Sources: `index.php` 364-424 `index.php` 446-544

Configuration Integration

Core components rely on a robust configuration system that provides settings from multiple sources:

1. Global configuration in `heuristConfigIni.php`
2. Instance-specific configuration in `configIni.php`
3. Database settings in the `sysIdentification` table
4. User preferences stored in the `ugr_Preferences` field of the `sysUGrps` table

The System class integrates these configuration sources to provide a unified configuration interface to all components.

Sources: `hserv/consts.php` 26-60 `hserv/System.php` 557-609

Conclusion

The core components of Heurist work together to provide a comprehensive knowledge management system. The System class serves as the central coordinator, with specialized components handling specific aspects of functionality such as record management, file handling, and user interface.

Understanding these core components and their interactions is essential for effectively working with and extending the Heurist codebase.